

AUTONOMOUS AGENTIC AI TRAFFIC MANAGEMENT & CONTROL CENTER

Comprehensive Technical Architecture, Multi-Agent Coordination, Full Source Code Walkthrough & Package Specification

Author / Architecture: Multi-Agent AI Engineering Team	Version: 2.4.0 (Production Release)
Target Domain: Smart City Intelligent Transport Systems (ITS)	Simulation Engine: Eclipse SUMO + TraCI API
Hardware Target: ESP32 Microcontrollers via MQTT Broker	Frontend: React 18, Vite 5, TailwindCSS

1. Executive Project Overview

The **Autonomous Agentic AI Traffic Management & Control Center** is an end-to-end cyber-physical Intelligent Transportation System (ITS). It dynamically mitigates urban traffic congestion, eliminates deadlock bottlenecks, prioritizes emergency first-responders (ambulances, police, fire engines), and coordinates hardware traffic light controllers in real-time.

The platform bridges high-fidelity micro-simulation (Eclipse SUMO via TraCI), autonomous multi-agent reinforcement decision loops, IoT edge hardware microcontrollers (ESP32 via MQTT), and an interactive 60 FPS browser-based digital twin control center.

2. System Architecture & High-Level Design

The system operates across four deeply integrated architectural layers:

- Layer 1 (Perception & Simulation Engine):** Continuous micro-simulation of a 4-way urban intersection (Junction J1) with real-time vehicle velocity, halting queues, and approach densities extracted via Python TraCI.
- Layer 2 (Multi-Agent Decision Pipeline):** A 4-agent asynchronous orchestration loop (Monitoring -> Analysis -> Optimization -> Supervisor) processing live telemetry and computing dynamic green allocations every 3 seconds.
- Layer 3 (IoT Actuation & Failsafe Communication):** Real-time MQTT message broker synchronizing signal status commands to physical ESP32 nodes controlling traffic light LEDs with sub-15ms round-trip latency.
- Layer 4 (Real-Time Digital Twin Dashboard):** React/Vite Single Page Application with WebSocket telemetry streaming, interactive scenario stress-testing, and dual-mode signal sequencing control.

3. Multi-Agent AI System Architecture

The decision intelligence is partitioned into four specialized autonomous agents executing in a sequential pipeline:

Agent Name	Role & Responsibilities	Key Algorithms / Functions
Agent 1: Monitoring Agent (Telemetry Ingestion)	Polls SUMO/camera providers, standardizes vehicle positions, counts, speeds, and queues.	Data normalization, schema validation, latency tracking (~5ms).

Agent 2: Analysis Agent (Congestion Analytics)	Evaluates junction density per km, determines busiest corridors, computes Level of Service (LOS A-F).	Traffic density classification, queue wait-time estimation, anomaly scoring.
Agent 3: Optimization Agent (Timing Plan Generator)	Calculates dynamic green signal durations prioritizing congested approaches and resolving turning conflicts.	Multi-factor weighted scoring formula + priority boost (15s to 75s range).
Agent 4: Supervisor Agent (Safety & Emergency Dispatch)	Enforces safety bounds (10s <= green <= 90s), activates Emergency Green Wave corridor preemption, dispatches MQTT.	Safety boundary validation, preemption override, MQTT command publication.

Mathematical Model for Adaptive Green Allocation:

```
For each junction approach $i$ in {\text{North, South, East, West}}$:
$$\text{BaseScore}_i = 0.35 \cdot \left(\frac{\text{Density}_i}{60}\right) + 0.30 \cdot \left(\frac{\text{Queue}_i}{50}\right) + 0.20 \cdot \left(\frac{\text{WaitTime}_i}{60}\right) + 0.15 \cdot \left(\frac{\text{Count}_i}{40}\right)$$
$$\text{FinalScore}_i = \min\left(1.0, \; \text{BaseScore}_i + (0.25 \cdot \text{ if Congested else } 0.0)\right)$$
$$\text{GreenDuration}_i = \max\left(15, \; \min\left(75, \; \lfloor 15 + 50 \cdot \text{FinalScore}_i \rfloor\right)\right)$$
```

4. Selectable Signal Sequencing Modes

The system provides a real-time toggle between two distinct traffic engineering control strategies:

Control Mode	Phase Progression	Engineering Advantages & Use Cases
Paired Corridors (2-Phase Mode)	Phase 1: North + South GREEN Phase 2: East + West GREEN	High Throughput: Discharges opposing traffic simultaneously. Best for balanced bidirectional flow and arterial throughput.
One-by-One Approach (4-Phase Isolated)	Phase 1: North GREEN only Phase 2: East GREEN only Phase 3: South GREEN only Phase 4: West GREEN only	Zero Turn Conflicts: Completely eliminates left/right turning collision risks. Allocates green time purely to demanding lanes.

5. Packages, Libraries & Dependencies Inventory

The project utilizes specialized libraries across Python (Backend), JavaScript/Node (Frontend), and C++ (ESP32 Firmware):

Package / Library	Version	Category	Technical Purpose & Why We Used It
fastapi	>=0.100.0	Backend	High-performance asynchronous REST API framework with native OpenAPI schema generation.
uvicorn[standard]	>=0.22.0	Backend	Lightning-fast ASGI production web server powered by uvloop and httptools.
pydantic	>=2.0.0	Backend	Data validation, strict typing, and schema serialization for all telemetry states.
websockets	>=11.0	Backend	Full-duplex WebSocket streaming for real-time telemetry, signal sync, and logs.
paho-mqtt	>=1.6.1	Backend	Lightweight MQTT communication client for hardware microcontroller bridging.
pandas	>=2.0.0	Backend	Time-series data manipulation, dataset rolling buffers, and trend calculations.
numpy	>=1.24.0	Backend	Vectorized numeric calculations, speed aggregation, and matrix transformations.
python-dotenv	>=1.0.0	Backend	Environment variable management (.env config parsing).

traci (SUMO)	1.20+	Simulation	Traffic Control Interface to interact with Eclipse SUMO simulation engine at runtime.
react & react-dom	^18.2.0	Frontend	Component-based reactive UI rendering with hooks, memos, and virtual DOM diffing.
vite	^5.2.0	Frontend	Next-gen frontend build tool with instant HMR and ES-module proxy routing.
@tanstack/react-query	^5.28.0	Frontend	Declarative server state management, query caching, and automatic refetching.
tailwindcss	^3.4.1	Frontend	Utility-first CSS framework for responsive layout and glassmorphism styling.
recharts	^2.12.3	Frontend	Declarative SVG charting library for volume and speed time-series visualizations.
framer-motion	^11.0.24	Frontend	Production-ready spring physics animations, layout transitions, and glows.
lucide-react	^0.359.0	Frontend	Modern iconography set with 40+ clean SVG icons.
zustand	^4.5.2	Frontend	Lightweight, boilerplate-free state store for scenarios, themes, and connection status.
axios	^1.6.8	Frontend	Promise-based HTTP client for API mutation calls and REST interactions.
PubSubClient	^2.8	Firmware	Arduino MQTT client enabling ESP32 subscription and publication on traffic topics.
ArduinoJson	^6.21.0	Firmware	Embedded JSON parsing for low-memory microcontrollers with zero-copy decoding.
WiFi.h	Built-in	Firmware	ESP32 802.11 b/g/n wireless networking stack.

6. Codebase File Structure & Module Purposes

Module / File Path	Primary Purpose & Code Implementation
backend/app/main.py	FastAPI app entrypoint, CORS setup, WebSocket routes (/ws/live, /ws/signals, /ws/logs, /ws/heartbeat), background pipeline runner.
backend/app/core/pipeline.py	4-agent sequential pipeline execution engine, decision generation, MQTT dispatch, and WebSocket state broadcasts.
backend/app/data_providers/sumo_provider.py	TraCI integration, 300ms simulation thread, telemetry extraction, 2-phase vs 4-phase sequencing, and scenario injection.
backend/app/data_providers/schemas.py	Authoritative Pydantic models for OverviewState, ApproachState, SignalHeadState, HardwareNodeState, and TrendData.
backend/app/agents/signal_optimization_agent.py	Dynamic AI green allocation algorithm, weighted scoring, congestion priority boosts, and 4-phase per-approach splits.
backend/app/agents/supervisor_agent.py	Safety validation (10s-90s), emergency corridor preemption overrides, and decision approval.
backend/app/api/routes_signals.py	REST API for signal status, emergency corridor activation, adaptive mode toggle, and GET/POST /signals/mode switcher.
backend/app/mqtt/mqtt_client.py	MQTT client connecting to broker, publishing traffic/signals/control, and receiving traffic/esp32/ack.
frontend/src/App.jsx	Root application layout mounting the 12 dashboard section panels with QueryClientProvider.
frontend/src/components/signals/SignalGrid.jsx	Active signal heads panel with interactive 2-Phase vs 4-Phase mode selector buttons.
frontend/src/components/simulation/IntersectionRenderer.jsx	Dynamic 2D SVG canvas simulation rendering road geometry, animated vehicles, turning trajectories, and stop bars.
frontend/src/components/simulation/ScenarioInjectorPanel.jsx	Stress-testing panel to inject Rush Hour, Emergency Corridor, Accident Blockage, and Adverse Weather.
frontend/src/components/prediction/AIPredictionPanel.jsx	AI predictive forecasting panel displaying +15m, +30m, and +60m congestion projections.
frontend/src/api/websocket.js	WebSocket connection manager with auto-reconnect, JSON frame parsing, and pub/sub event subscription.
firmware/esp32_traffic_controller.ino	Arduino C++ firmware for ESP32 with WiFi, MQTT client, GPIO LED driving, and 10s failsafe fallback timer.

7. ESP32 Hardware Pinout & MQTT Communication

Each physical traffic light cluster at Junction J1 is actuated via ESP32 GPIO pins based on JSON payloads received from the MQTT broker:

Hardware Node ID	Direction Served	Green / Yellow Pin	Red Pin	MQTT Topic
ESP32-J1-N	North Approach	GPIO 18	GPIO 19	traffic/signals/SIG-N1
ESP32-J1-E	East Approach	GPIO 21	GPIO 22	traffic/signals/SIG-E1
ESP32-J1-S	South Approach	GPIO 25	GPIO 26	traffic/signals/SIG-S1
ESP32-J1-W	West Approach	GPIO 32	GPIO 33	traffic/signals/SIG-W1

8. Operational Startup & Execution Instructions

To launch the entire system in a local development or production environment:

Step 1: Start the Backend (FastAPI + Uvicorn):

```
cd "c:\Traffic Project\traffic-control-center\backend"  
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8001 --reload
```

Step 2: Start the Frontend (Vite + React):

```
cd "c:\Traffic Project\traffic-control-center\frontend"  
npm run dev
```

Step 3: Access the Control Center Dashboard:

Open browser at **<http://localhost:3000>** (Swagger API Docs at **<http://localhost:8001/docs>**).